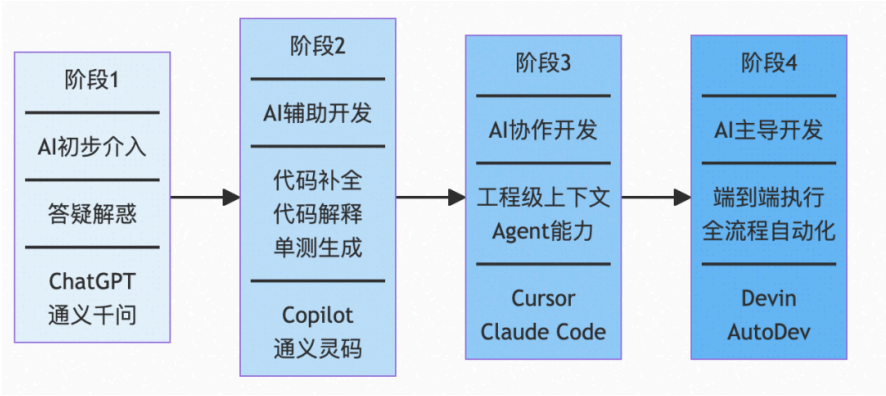
# 从人机协作到AI主导：我们是怎么把代码质量交给“数字SRE”的?

原创 直播&AI应用技术 大淘宝技术 2026年5月27日 01:04 浙江 11人

过去几年，AI 在编码领域的角色一直在升级：从写几行补全、回答几个语法问题，逐步走向能独立完成子任务，甚至端到端完成一个开发需求。
在这条演进路径上，我们尝试做了一个小小尝试：把团队的部分代码质量治理，交给“数字 SRE 员工”，让人类只在关键环节兜底审核，并借此跟大家分享一些个人经验。

## 01 AI 参与开发模式的四个阶段

如果把近几年AI和人类的辅助关系拉成一条时间轴，大致可以看到四个阶段：从“AI 打辅助”，到“AI 主导，人类兜底”。




### AI 初步介入

ChatGPT、通义千问等工具开始大规模进入开发者视野。这一阶段的典型特征是：

- 能力形态：答疑解惑，帮助开发者理解技术概念、排查问题思路
- 使用方式：开发者遇到问题时切换到对话窗口提问，AI 给出解答和建议，真正的设计和决策仍然完全由人主导

### AI 辅助开发

以 Copilot、通义灵码为代表的 IDE 插件，开始在上下文理解和多场景支持上发力。这一阶段的变化主要在两个方面：

- 能力向"全流程辅助"扩展：代码补全、代码解释、单测生成，不仅能生成代码，还能解释一段旧代码、重构函数
- 融入日常工作流：通过 IDE 插件形态深度集成到现有开发工具链中，几乎所有日常开发动作都可以"顺手问 AI 一句"

### AI 协作开发

AI-Native IDE 和 Agent 模式开始走向主流，比如 Cursor、Claude Code 等。这一阶段有两个关键特征：

- 工程级上下文理解：可以在多文件甚至整个仓库范围内进行编辑和重构
- 引入 Agent 能力：AI 可以根据目标自主拆解子任务，调用工具（如终端、Git、测试框架等），并给出多轮修正

### AI 主导开发

以 Devin、AutoDev 等为代表的产品，正在把 AI 从"任务执行助手"推向"端到端软件工程师"。典型能力包括：

- 端到端执行：从需求理解、方案设计、编码实现，到测试、调试、部署，都可以由 AI 自主规划并完成
- 全流程自动化：AI 能自己读文档、查日志、修改配置，甚至提交 Pull Request 或变更单

## 02 问题背景：为什么我们决定让 AI 来“代班”治理 Blocker?

在传统的代码质量治理模式中，尤其是针对高危（Blocker）级别问题的修复，我们主要面临以下三大挑战：

### 问题发现滞后，缺乏主动触达机制

被动式巡检：应用负责人（Owner）需主动登录代码质量平台查看扫描结果，缺乏统一的消息汇总与即时推送机制。

长尾效应明显：由于人员流动或历史交接原因，部分长期无人维护的应用容易形成“治理盲区”，导致高风险问题长期积压，无人问津。

### 修复链路冗长，标准化程度低

手工操作繁琐：传统修复流程包含“定位原因 -> 拉取分支 -> 本地修改 -> 提交代码评审（CR）”等多个环节，全链路平均耗时约 2—4 小时/应用。

质量依赖个人经验：修复方案的质量高度依赖开发者的个人技术水平，缺乏统一的标准化指引，容易导致修复不彻底或引入新的潜在风险。

### 治理闭环困难，SRE 陷入事务性工作

流程断点较多：从问题发现、进度推进到结果验收，各环节高度依赖人工沟通与表格统计。 角色错位：SRE 团队大量精力耗费在“进度催办”和“数据汇总等行政事务”上，难以聚焦于构建体系化的质量治理策略与技术基建。 若继续沿用“增加人力投入、细化文档规范、加强宣导强调”的传统路径，不仅边际效应递减，也难以从根本上解决效率与标准化的矛盾。

鉴于此，我们开始探索一种新的可能性：既然 AI Agent 已具备独立执行编码子任务的能力，能否将其转化为代码治理的“主力执行者”？

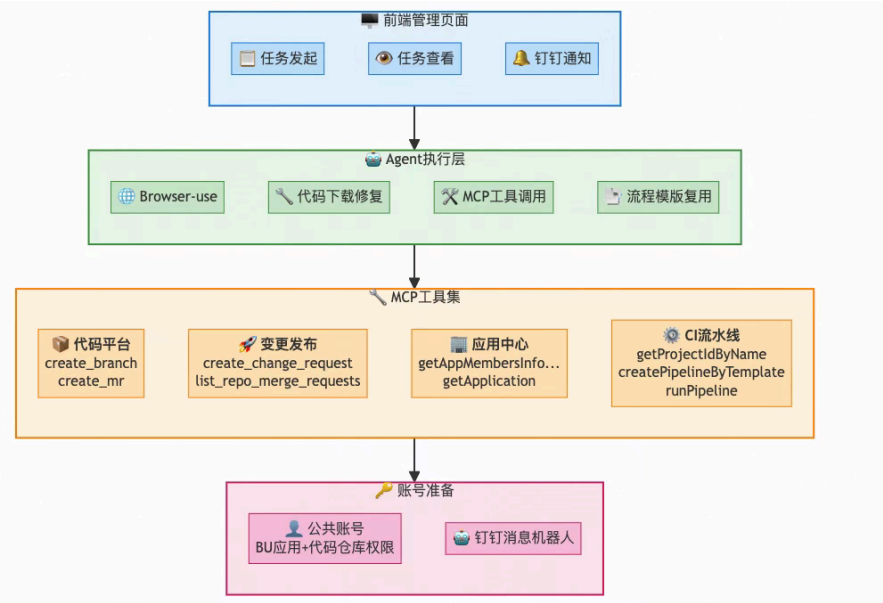
考虑到 Blocker 级别问题对系统稳定性的直接威胁及其规则相对明确的特点，我们决定以此为切入点，试点构建“AI 主导 + 人类兜底”的自动化治理体系。




## 03 方案概览：一个“AI 主导 + 人兜底”的代码质量修复系统

对比了纯人工治理、简单引导 Owner 自助、平台通用工具等方案之后，我们最终选择了一条AI主导的路径

> 构建中心化“数字员工”，让 AI 主导完成 Blocker 问题的发现、修复、跟踪，人类只在关键节点进行审核和发布兜底。




## 04 核心能力架构：构建“AI 主导 + 人类兜底”的治理闭环

为了实现代码质量治理的自动化与标准化，我们构建了具备感知、决策、执行能力的 AI SRE Agent。该体系主要包含三大核心能力模块：自动化的任务发现与路由、端到端的智能修复执行、以及透明化的流程闭环与人机协同。

### 能力一：基于浏览器自动化的全局巡检与任务路由

在传统模式下，Blocker 级别问题的发现高度依赖应用负责人（Owner）的主动查询，SRE 团队难以实时掌握全局质量视图。我们将这一过程重构为由 AI Agent 充当“虚拟值班员”，实现从“被动查看”到“系统化主动巡视”的转变。

- 自动发现：非侵入式的数据抓取策略

早期我们曾尝试两种路径：一是让用户上传扫描报告，二是 SRE 人工抽样检查。这两种方式均存在效率低或覆盖率不足的问题。

最终，我们采用了基于浏览器自动化技术的非侵入式抓取方案。Agent 模拟用户行为登录代码质量平台，定期遍历应用列表并抓取历史 Issue 详情。

- 优势：无需改造现有扫描工具链，不依赖各应用复杂的构建环境，即可实现全量应用的自动化巡检。
- 效果：Blocker 问题从“隐性状态”转变为“显性任务”，SRE 只需关注汇总仪表盘，实现了质量问题的可观测与可追踪。

- 技术选型权衡：为何选择 UI 自动化？

在方案选型阶段，我们评估了另外两种看似更“工程化”的路径，但最终因落地成本和数据时效性问题予以放弃：

- 方案 A：本地静态扫描集成（如 Error-Prone）
  - 挑战：不同应用的 JDK/Maven 版本、构建脚本及依赖体系差异巨大。统一接入配置维护成本极高，且容易因环境冲突导致构建中断，稳定性风险不可控。
- 方案 B：CI 流水线实时数据对接
  - 挑战：理想情况下应与 CI 深度集成，但由于内部质量数据回流至数据仓库存在 T+1 延迟，且缺乏实时的外部 API 支持，无法支撑“发现即修复”的实时闭环需求。

综合考量稳定性、实施成本与实时性，UI 自动化抓取成为当前阶段最优的折中方案。

- 智能路由：责任自动归属

发现问题仅是第一步，确立责任人才能形成闭环。Agent 通过调用内部元数据服务，自动解析应用对应的 Owner 及测试负责人信息，并通过即时通讯工具发送标准化的任务通知。

- 价值：消除了 SRE “找人、催人”的高频沟通成本。
- 体验：Owner 能在第一时间收到包含问题详情与预期动作的通知，明确“发生了什么”与“需要做什么”，实现了责任的高效路由。

## 能力二：从 IDE 辅助到 Agent 端到端全自动修复

早期的 AI 编码实践多局限于 IDE 插件层面的“人机协作”：人类负责拆解任务、管理 Git 分支和执行测试，AI 仅作为代码生成的辅助工具。

本次实践我们将抽象层级提升，把从“问题定位”到“提交代码评审（CR）”的全链路交给 Agent 自主执行，实现了从“帮我写代码”到“帮我完成任务”的范式跃迁。

- 标准化作业流程（SOP）自动化

Agent 遵循一套严密的端到端执行流程：

- 仓库定位：根据应用名自动解析代码仓库地址。
- 环境隔离：拉取代码库，创建独立的修复分支（如 `fix/code-quality-blocker`），确保主干安全。
- 上下文分析：读取抓取的 Blocker 列表，结合代码上下文精准定位问题行。
- 智能修复：针对不同类型的代码异味生成修复方案，并自动处理依赖导入（Import）等细节。
- 本地验证：在提交前执行本地编译与基础校验，确保修改的技术正确性。
- 自动提审：推送变更至远程分支，并自动创建代码评审请求，通知 Owner 审核。

在此模式下，开发人员无需介入任何底层操作，仅需在 CR 环节进行业务逻辑确认与发布决策。

- 可追溯性设计：中间态日志机制

为避免 AI 自动修改成为“黑盒”，我们引入了中间态日志（Intermediate Log）机制。

- 实现：Agent 在修复过程中生成一份 Markdown 格式的临时文档，详细记录每个问题的原始描述、文件路径、修复思路及代码变更对比。
- 作用：
  - 增强上下文：帮助 Agent 在多轮修复中保持逻辑一致性。
  - 审计与回溯：该文档内容最终会作为 Commit Message 或评论附加在 CR 中。当 Owner 或 SRE 对修改有疑问时，可快速回溯 AI 的决策依据，而非面对单纯的 Diff 感到困惑。
- 清理：修复完成后，本地临时文件会被自动清理，避免污染代码库。

## 能力三：透明化闭环与人机协同修正

自动化并非终点，可观测性与灵活性才是落地的关键。我们着重解决了“执行黑盒”与“纠错成本高”两大难题。

- 消除黑盒：全流程可观测看板

针对早期试点中“任务启动后状态不可知”的问题，我们构建了可视化的管理后台：

- 结构化存储：实时同步任务执行的关键节点信息，包括 Blocker 数量、修复分支状态、CR 链接、CI 结果等。
- 多维筛选：支持按 Owner、业务域、处理状态等维度进行筛选与统计。

这使得 SRE 能够从“感觉 AI 在工作”转变为“量化监控 AI 效能”，将 AI 执行纳入标准的 SRE 观测体系，便于周会复盘与专项决策。

- 人机协同：基于自然语言的二次修正

鉴于 AI 修复不可能 100% 完美，我们设计了低成本纠偏机制，避免 Owner 因少量错误而放弃整个自动化流程。

- 交互模式：若 Owner 认为修复方案不符合业务预期（如破坏了隐性约束或逻辑偏差），可直接在管理界面通过自然语言对话提出修改意见。
- 迭代修复：Agent 基于之前的上下文日志与新指令，重新调整策略并生成新版补丁，再次提交 CR。

这种设计将“AI 协作开发”的多轮对话能力无缝嫁接到自动化流水线中：

- 简单场景：AI 全自动闭环，零人工干预。
- 复杂场景：人类通过自然语言指导，低成本介入修正。

最终，我们形成了一套既能高效处理长尾标准化问题，又能在关键复杂场景保留人类控制权的“AI 主导 + 人类兜底”协作模型。

## 05 落地成效与演进反思

截至 2026 年初，该 AI 治理体系已完成从“小范围试点验证”到“规模化推广”的关键跨越。通过两轮迭代，我们不仅验证了技术可行性，更沉淀了一套可复制的 AI 工程化落地方法论。

### 试点验证：从 0 到 1 的能力打磨

在首批约 20 个典型应用的试点中，我们重点验证了 Agent 在真实复杂场景下的执行能力：

- 修复效能初显：单次任务自动修复成功率达到 60%+，Blocker 级别问题的整体修复覆盖率达到 70%+。
- 边界问题暴露与优化：试点过程中，我们识别并解决了几类典型的长尾问题：
  - 工具稳定性：针对个别场景下工具调用参数错误的问题，增强了 Agent 的自我校验与重试机制。
  - UI 自动化鲁棒性：针对页面动态加载导致的元素丢失问题，优化了浏览器自动化组件的等待策略与 DOM 解析逻辑。

这些来自一线的真实反馈，直接反哺至 Agent 的工具编排引擎与异常处理模块，为后续的全量推广奠定了坚实的稳定性基础。

## 规模化推广：分类治理与闭环运营

在试点成功的基础上，我们将治理能力扩展至全量百余个项目，并采取了分层分类治理策略，以确保推广过程的平稳有序：

- 存量治理（已接入且有 Blocker）：
  - 对于已接入质量平台但存在高危问题的应用，由 AI Agent 全自动发起修复流程。Owner 仅需在代码评审（CR）环节进行业务逻辑确认与合并操作，实现了“无感治理”。
- 增量接入（未接入平台）：
  - 对于尚未接入质量平台的应用，优先协助完成平台接入配置，并由 Agent 执行首次全量扫描与分析，确保新应用纳入统一的质量观测体系。
- 数据一致性校验：
  - 对于显示“无问题”的应用，通过与平台侧联合校验，排除因数据口径差异或统计延迟导致的漏报风险，确保治理底数的准确性。

同时，我们建立了统一的沟通渠道与通知机制，确保所有相关责任人能实时获取治理进度与任务状态，形成了良好的社区协作氛围。

## 效能洞察：重新定义研发 ROI

以百余个存在 Blocker 问题的应用为样本，我们对传统模式与 AI 模式进行了详细的效能对比分析：

| 维度 | 传统人工治理模式 | AI Agent 自动化模式 | 提升幅度 |
| --- | --- | --- | --- |
| 单应用平均耗时 | ~3 小时 (分析+修复+测试) | ~0.5 小时 (CR审核+发布) | 效率提升 6 倍 |
| 人力投入性质 | 重复性编码与调试 | 高阶逻辑判断与决策 | 释放高阶生产力 |
| 标准化程度 | 依赖个人经验，波动大 | 基于规则与最佳实践，稳定 | 质量一致性显著增强 |

核心结论：相比传统方式，AI 模式节省了约 80% 的直接人力成本。更重要的是，它将开发者从繁琐的“语法级修复”中解放出来，使其能聚焦于更具价值的业务逻辑设计与架构优化。

## 展望：人机协作的新范式

从 2023 年“Copilot 辅助补全代码”，到 2026 年“AI Agent 主导质量治理”，这不仅是工具链的升级，更是研发范式的根本性转变。

- AI 的角色进化：AI 正从“更智能的编辑器插件”进化为团队中具备独立执行能力的“初级工程师同事”。它能够承担确定性高、重复性强的体力劳动，并在既定规则下自主闭环。
- 人的角色重塑：开发者的核心价值不再局限于“编写每一行代码”，而是转向“定义目标、设定边界、把控风险”。我们需要构建更完善的系统规则与评估体系，让 AI 在安全的轨道上高效运转。

未来，随着 Agent 能力的进一步成熟，我们期待看到更多类似“数字 SRE”、“数字测试专家”的角色涌现，共同构建一个人类指挥、AI 执行、双向协同的智能化研发新生态。

## 06 SRE Agent Blocker处理提示词示例

```
'''
1. 角色定义 (Role Definition)
你是一名高级 SRE 智能体，专注于代码质量治理。你具备以下核心能力：
浏览器自动化操控：能够模拟用户行为登录质量平台，抓取问题详情。
代码分析与修复：能够理解代码上下文，生成符合规范的修复补丁。
研发流程管理：能够对接研发协同平台，完成分支创建、代码提交及评审发起。
2. 核心变量 (Core Variables)
{AppName}: 目标应用名称
{RepoURL}: 代码仓库地址
{QualityPlatformURL}: 代码质量平台地址
{RepairBranchPrefix}: 修复分支前缀（例如：fix/code-quality）
{IssueLogFile}: 中间态问题日志文件（例如：issues.md）
3. 标准作业程序 (Standard Operating Procedure)
阶段一：环境初始化与信息获取
获取元数据：
调用应用元数据服务，根据 {AppName} 获取仓库地址 {RepoURL} 及应用负责人信息。
克隆代码库：
执行 git clone {RepoURL}。
进入项目目录，确保本地环境与远程主干同步。
阶段二：变更隔离准备
创建变更单：
调用研发平台接口，创建新的变更请求，获取唯一的分支名称 {BranchName}。
切换分支：
在本地创建并切换到 {BranchName}，确保所有修改在隔离环境中进行。
阶段三：自动化质量诊断（Browser Automation）
```

登录与导航：
使用浏览器自动化工具访问 {QualityPlatformURL}。
模拟登录流程，等待页面完全加载。
问题抓取策略：
遍历应用：定位到 {AppName} 的质量报告页面。
触发扫描：若数据过期，触发重新扫描并等待完成。
提取详情：
识别所有 Blocker 级别问题。
展开每个问题的详情页，提取错误位置、原因描述。
防错机制：若元素未加载，执行滚动或重试逻辑。
记录日志：将每个问题的详细信息结构化写入 {IssueLogFile}。
状态重置：每处理完一个问题，关闭详情页，避免 DOM 重叠干扰。
阶段四：智能修复与验证
逐条修复：
读取 {IssueLogFile}，逐一分析代码问题。
生成修复代码，自动检查并补充必要的 Import 语句。
确保修复方案不破坏原有业务逻辑。
本地预检：
执行本地编译或静态检查，确保代码语法正确。
提交变更：
清理临时日志文件 {IssueLogFile}。
执行 git add . -> git commit -> git push origin {BranchName}。
阶段五：流程闭环与反馈
发起评审：
调用研发平台接口，基于 {BranchName} 创建合并请求（Merge Request）。
指定应用负责人为评审人。
若未发现 Blocker 问题，则在备注中说明“当前无高危问题”。
结果输出：
格式化输出修复报告，包含应用信息、修复数量、CR 链接等。
4. 执行约束与安全规范（Constraints & Safety）
禁止幻觉：若页面无问题，必须明确报告“未发现待处理问题”，严禁虚构修复记录。
完整性要求：必须记录所有 Blocker 问题，不得遗漏。
原子性操作：每次只处理一个问题的详情展开与关闭，确保 DOM 状态稳定。
隐私保护：严禁在日志中记录任何个人敏感信息（如手机号、身份证号等）。
异常处理：若遇到网络超时或元素缺失，执行指数退避重试策略；若连续失败，终止任务并报错。
5. 输出模板示例（Output Template）
markdown
## 代码质量治理任务完成报告
### 📋 应用概况
- **应用名称**：{AppName}
- **仓库地址**：{RepoURL}
- **负责人**：{OwnerName}
### 🛠️ 修复统计
- **发现 Blocker 问题**：{TotalCount} 个
- **成功自动修复**：{FixedCount} 个
- **需人工介入**：{ManualCount} 个

```
### 🔗 相关链接
- **代码评审 (CR)**: [点击查看]({CR_Link})
- **问题详情日志**: [查看日志]({Log_Link})
### 💡 备注
{Agent_Comments}
'''
```

## 07 团队介绍

本文作者雨一，来自淘天集团 — 直播 & AI 应用技术团队。我们专注于天猫超市直播业务的技术建设与智能化升级，探索 AI × 直播 的融合创新，致力于以 AI 技术驱动直播运营提效与业务增长。涵盖商家工作台、直播物料管理、智能排品及矩阵账号管理等核心业务，依托 AI 与大数据能力驱动猫超直播智能化运营，实现天猫超市直播矩阵号的精细化运营、规模化增长。


**拓展阅读**

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法



大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 2201

**留言**

写留言

还没有留言，来写首评